

Project Technical Note

Parameterized PINN for Current-Based Inference in a 1D PNP Model

Jack Jia

May 31, 2026

Short Version

This project builds a parameterized PINN surrogate for a one-dimensional Poisson–Nernst–Planck (PNP) model. The trained network takes

$$(x, t, D_+, D_-)$$

as input and predicts

$$(c_+, c_-, \phi),$$

where c_+ and c_- are ion concentrations, ϕ is electric potential, and D_+, D_- are the diffusion parameters we want to infer.

The current goal is not just to solve the PDE once. The goal is to use the PINN as a fast reusable forward model inside an inverse problem. We generate synthetic electrode charging-current data from an independent finite-difference reference solver, add Gaussian noise, and then use the trained PINN to infer the posterior distribution of D_+ and D_- .

What I Am Modeling

The physical model is a nondimensional 1D binary-electrolyte PNP system on

$$x \in [0, 1], \quad t \in [0, 0.2], \quad D_+, D_- \in [0.5, 2.0].$$

The two ion fluxes are

$$\begin{aligned} J_+ &= -D_+ \partial_x c_+ - D_+ z_+ c_+ \partial_x \phi, \\ J_- &= -D_- \partial_x c_- - D_- z_- c_- \partial_x \phi, \end{aligned}$$

with

$$z_+ = 1, \quad z_- = -1, \quad \epsilon = 1.$$

The PDE residuals are:

$$\begin{aligned} \partial_t c_+ + \partial_x J_+ &= 0, \\ \partial_t c_- + \partial_x J_- &= 0, \\ -\epsilon \partial_{xx} \phi - (z_+ c_+ + z_- c_-) &= 0. \end{aligned}$$

The initial and boundary conditions are:

$$\begin{aligned} c_+(x, 0) &= 1, & c_-(x, 0) &= 1, \\ \phi(0, t) &= -0.5, & \phi(1, t) &= 0.5, \\ J_+(0, t) &= J_+(1, t) = 0, & J_-(0, t) &= J_-(1, t) = 0. \end{aligned}$$

This is a blocking-electrode setup. Ions do not pass through the boundaries. The current signal used later is therefore charging current, not ionic flux through the wall.

Why This Setup Is Reasonable

This is a clean benchmark model, not a full experimental electrochemical cell. The no-flux ion boundary is not arbitrary: it corresponds to blocking electrodes, where the electrolyte charges capacitively but there is no Faradaic reaction at the wall.

The most idealized part is the electrode interface. In a real experiment, the interface may include Stern-layer capacitance, Faradaic kinetics, adsorption, leakage current, and external circuit effects. I am not modeling those yet. The current benchmark is meant to test whether a trained PINN surrogate can support a controlled current-based inverse problem.

What The PINN Learns

The network learns the parameterized map

$$(x, t, D_+, D_-) \mapsto (c_+, c_-, \phi).$$

The active model is an Equinox MLP:

input dimension	4
output dimension	3
width	256
depth	8
activation	tanh

The model uses hard constraints for the most important fixed-value conditions. For concentration:

$$c_+ = 1 + t r_+, \quad c_- = 1 + t r_-.$$

So the concentration initial condition is exact.

For electric potential:

$$\phi = \phi_{\text{wall}}(x) + 4\xi(1 - \xi)r_\phi.$$

The factor $4\xi(1 - \xi)$ is zero at both walls, so the wall voltages are exact.

How The PINN Is Trained

Each training step samples three groups of collocation points:

- interior domain points for the PNP residuals,
- boundary-layer points near the two walls,
- wall points for no-flux boundary residuals.

The loss has three groups:

$$\mathcal{L} = w_{\text{dom}}\mathcal{L}_{\text{dom}} + w_{\text{bl}}\mathcal{L}_{\text{bl}} + w_{\text{bc}}\mathcal{L}_{\text{bc}}.$$

The group weights are adjusted during training with an NTK-style balancing rule. Training uses Adam on the CPU backend. CPU is used because the nested automatic differentiation in this PINN is stable there on the local Apple M4 environment.

Current Trained Run

The main trained run is:

checkpoints/runs/pnp_1d_hard_phi_high_quality_20260527_074537

It completed 500,000 steps. The main training summary is:

Metric	Value
Final loss	1.236948×10^{-2}
Final domain loss	2.227460×10^{-3}
Final boundary-layer loss	5.163684×10^{-3}
Final boundary loss	7.020860×10^{-3}
Best logged loss	3.645345×10^{-3}
Final step	500,000
Training time	about 8 h 9 m

How I Check The Forward Model

There are two levels of forward diagnostics.

Phase A: PINN-only checks

These checks evaluate whether the trained PINN obeys the constraints and basic physical sanity checks.

Check	Result
Wall-voltage error	0
$c_+(x, 0) - 1$ max error	0
$c_-(x, 0) - 1$ max error	0
Minimum c_+ on diagnostic grid	0.5867
Minimum c_- on diagnostic grid	0.5894
Maximum c_+ mass drift	1.61×10^{-3}
Maximum c_- mass drift	3.25×10^{-3}

Phase B: comparison to reference solver

The reference solver is independent of the PINN. It is a finite-difference method-of-lines solver:

- Poisson’s equation is solved by a banded linear solve.
- The ion fluxes are computed on spatial faces.
- Time integration uses SciPy’s BDF solver.

At $n_x = 401$, the PINN-reference comparison is:

D_+	D_-	c_+ rel. L^2	c_- rel. L^2	ϕ rel. L^2
1.25	1.25	8.493×10^{-4}	5.090×10^{-4}	2.499×10^{-2}
0.50	0.50	1.575×10^{-3}	1.186×10^{-3}	1.689×10^{-2}
2.00	2.00	8.223×10^{-4}	6.199×10^{-4}	2.796×10^{-2}
0.50	2.00	3.304×10^{-3}	2.580×10^{-3}	2.604×10^{-2}
2.00	0.50	5.664×10^{-4}	1.080×10^{-3}	2.675×10^{-2}

The concentrations match very well. The potential error is larger, but still small enough for the current inverse demonstration.

The Inverse Problem

The inverse problem is:

Given noisy current measurements, infer the posterior distribution of D_+ and D_- .

The workflow is:

1. Pick true parameters, here $D_+ = 1.25$, $D_- = 1.25$.
2. Use the finite-difference/BDF reference solver to generate clean charging-current data.
3. Add independent Gaussian noise.
4. Use the trained PINN to predict current for candidate parameter pairs.
5. Compare predicted current to observed current through a Gaussian likelihood.
6. Normalize the likelihood over a parameter grid to obtain the posterior.

What Current Means Here

Because the boundary is no-flux, the boundary ionic flux current is zero. That is not the current used here.

The current used here is electrode charging current. First compute the electrode charge response $Q(t)$, then use interval-averaged current:

$$I_k = \frac{Q(t_{k+1}) - Q(t_k)}{t_{k+1} - t_k}.$$

For the exact PNP solution, electrode charge can be written in terms of the wall electric field. For the PINN evaluation, I use an equivalent Gauss-law integral over charge density. This is more stable because it avoids taking a boundary derivative of the neural-network potential.

Likelihood

The observation model is:

$$I_k^{\text{obs}} = I_k^{\text{PINN}}(D_+, D_-) + \varepsilon_k,$$

where

$$\varepsilon_k \sim \mathcal{N}(0, \sigma^2).$$

For this synthetic experiment:

$$\sigma = 0.02 \times \max_k |I_k^{\text{clean}}|.$$

In this run:

$$\sigma = 0.0418868.$$

So the likelihood is:

$$p(I^{\text{obs}} \mid D_+, D_-) = \prod_k \mathcal{N}\left(I_k^{\text{obs}}; I_k^{\text{PINN}}(D_+, D_-), \sigma^2\right).$$

The prior is uniform on:

$$D_+, D_- \in [0.5, 2.0].$$

The posterior is evaluated on a 41×41 grid.

Current-Based Inference Result

The current-based inference output is:

checkpoints/runs/pnp_1d_hard_phi_high_quality_20260527_074537/
diagnostics/phase_c_current_probe

The result is:

Quantity	Value
True D_+	1.25
True D_-	1.25
MAP D_+	1.25
MAP D_-	1.2125
Posterior mean D_+	1.23531
Posterior mean D_-	1.23677
95% interval for D_+	[1.0625, 1.4]
95% interval for D_-	[1.0625, 1.4375]
True value covered	yes

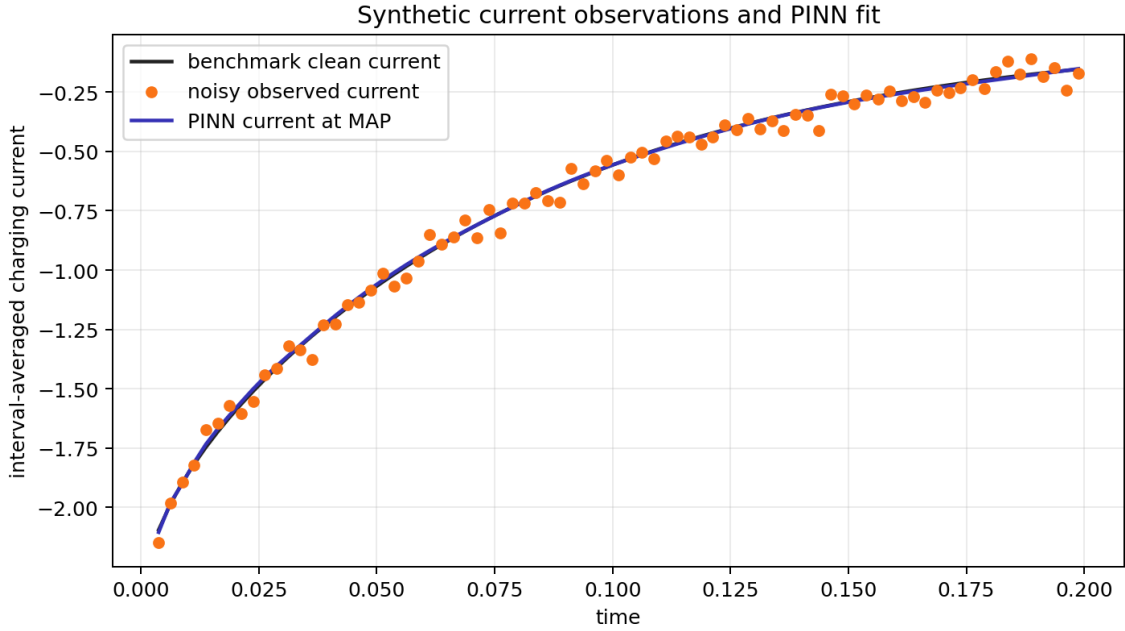


Figure 1: Clean benchmark current, noisy synthetic observations, and PINN current at the posterior MAP.

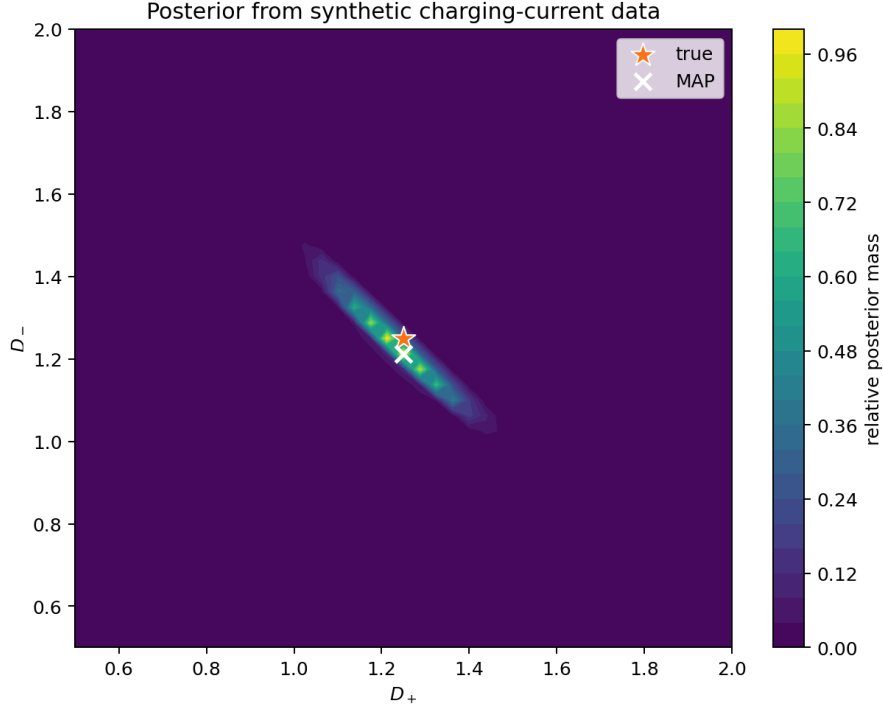


Figure 2: Posterior over D_+ and D_- from synthetic charging-current observations.

What This Shows

The main point is that direct concentration observations are not required for the inverse problem. A compressed electrical signal, here charging current, can still identify the diffusion parameters with uncertainty.

The posterior is not a point. It is an elongated region, which is useful: current measurements identify a combination of D_+ and D_- more strongly than each parameter independently. This is exactly the kind of uncertainty statement that a posterior distribution should reveal.

Why The PINN Is Useful

The PINN is expensive to train, but cheap to reuse. The main trained run took about 8 h 9 m. After that, repeated current predictions are much faster than solving the reference PDE repeatedly.

Measured on this setup:

Computation	Approximate time
One finite-difference/BDF solve	0.177 s
One PINN current evaluation in grid posterior	0.004–0.006 s
Full 41×41 current posterior grid	7.5 s

For one prediction, the PINN is not worth the training cost. For many likelihood evaluations, the cost can be amortized.

Limitations

The important limitations are:

- The model is 1D.
- The electrode interface is idealized.
- There are no Faradaic reactions.
- The current data are synthetic.
- The noise model is iid Gaussian.
- The posterior is computed on a grid rather than by MCMC.

These are acceptable for the current benchmark. The most natural extensions are Stern-layer boundary conditions, Faradaic boundary kinetics, richer current experiments, correlated noise, and MCMC posterior sampling.

Files To Check

The most useful project files are:

- `README.md`
- `src/pnp_pinn/model.py`
- `src/pnp_pinn/reference_solver.py`
- `src/pnp_pinn/current_observable.py`
- `scripts/run_current_probe.py`
- the run-level `RUN_SUMMARY.md`
- the current-probe `diagnostics/phase_c_current_probe/REPORT.md`

Bottom Line

This project currently demonstrates a complete controlled pipeline:

train a parameterized PNP PINN, validate it against an independent numerical solver, generate synthetic charging-current data, and infer D_+ , D_- through a posterior distribution using the PINN as a fast forward surrogate.